

1. Intro

Systém distribuovaného podpisovania blockchainových transakcií by mal vo veľmi zjednodušenej forme fungovať nasledovne: systém má na vstupe nepodpísanú transakciu a na výstupe poskytuje transakciu podpísanú tak, akoby bola podpísaná jedným privátnym kľúčom, avšak v skutočnosti podpis vzniká distribuovaným spôsobom.

Demonštračný prototyp implementovaný pre tento projekt ukazuje zmenšený model jedného z možných scenárov, akým by takýto systém mohol byť naprogramovaný v praxi pri custodial riešeniach typu **Fireblocks**¹ s využitím open-source technológií a knižníc. Tento projekt je demonštrovaný s použitím lokálneho simulovaného prostredia, avšak so zreteľom na to, že každá jedna použitá a implementovaná komponenta, či využitý dizajn v prototypu by v reálnom produkčnom prostredí išiel abstrahovať a implementovať iným, production-ready spôsobom. Tam, kde to považujeme za dôležité, tak to explicitne zdôrazníme, že ako by sa dala konkrétna časť projektu potenciálne nahradiť produkčnou variantou.

2. Research

Impulzom pre projekt bola snaha dosiahnuť úroveň bezpečnosti a praktickej použiteľnosti, ktorú v oblasti správy privátnych kľúčov pre digitálne aktíva ponúkajú komerčné riešenia postavené na **multi-party computation (MPC)**², typicky reprezentované platformou *Fireblocks*. Takéto systémy umožňujú realizovať podpisovanie transakcií bez toho, aby jediná entita niekedy vlastnila celý privátny kľúč, a zároveň poskytujú integráciu do infraštruktúry custodial služieb. Kľúčovou motiváciou bolo vytvoriť otvorenú, auditovateľnú a rozšíriteľnú alternatívu, ktorá by bola vhodná pre interné nasadenie a zároveň jej základ použiteľný pre prototypovú aplikáciu.

Významnú časť projektu tvoril samotný výskum dostupných protokolov a existujúcich implementácií. Na začiatku práce nebolo jasné, ktoré protokoly sú v praxi použiteľné, aké existujú open-source implementácie, ktoré blockchajny sú kompatibilné z hľadiska podpisových schém, ani čo bude potrebné doprogramovať nad rámec existujúcich knižníc. Z tohto dôvodu bol výskum vedený systematicky a tvoril podstatnú časť celkového úsilia projektu (odhadom približne polovicu času venovaného projektu). Výstupom výskumu nebol len výber konkrétnej knižnice, ale aj súbor technických rozhodnutí, ktoré následne ovplyvnili architektúru a dizajn implementácie, vzhľadom na použitú knižnicu.

¹ <https://www.fireblocks.com/>

² <https://www.fireblocks.com/what-is-mpc>

V úvodnej fáze výskumu sme sa zamerali na teoretické pozadie MPC protokolov a na to, aké kryptografické primitíva sa v tejto doméne používajú. Postupne sa ukázalo, že z praktického pohľadu najlepšie zodpovedá našim požiadavkám kryptosystém **threshold signatures (TSS)** ³, ktorý umožňuje vytvoriť podpis ekvivalentný štandardnému „single-signature“ podpisu, pričom podpis vzniká kooperáciou viacerých strán. Štandardne sa threshold signatures používajú v systémoch (t,n) -threshold, tzn. aby sme efektívne získali podpis, tak potrebujeme, aby t z n strán kooperovali. V našom prípade, ako aj ďalej ukážeme, si však vystačíme s tzv. **“non-threshold signatures”**, a to sú také threshold signatures, ktoré sú zo systému (n,n) -threshold. Jeden z hlavných argumentov, prečo nám stačia (n,n) -threshold je to, že systém máme ako custodian pod našou kontrolou, takže aj prípadne výpadky sú pod našou kontrolou. Viac k tomuto sa však budeme venovať v kapitole *Architektúra*.

Keďže projekt od začiatku cieľil primárne na podporu Bitcoinu a Etherea, kľúčovým kritériom bolo nájsť podpisovú schému, ktorá je spoločná pre oba ekosystémy a umožní multichainové použitie bez zásadných zmien kryptografického jadra. Analýza compatibility blockchainov následne viedla k výberu **ECDSA** nad krivkou **secp256k1** ako prirodzeného prieniku: **Ethereum** používa ECDSA na úrovni transakčného podpisu a **Bitcoin** ho používa pre adresy typu **legacy** a **SegWit**. Tento výber nám poskytuje natívnu podporu pre BTC aj ETH bez potreby meniť podpisovú schému medzi chainmi.

Zároveň sme zohľadnili aj **Taproot**. Posielať z legacy/SegWit adres na Taproot možné je. Jediné obmedzenie relevantné pre nás, ktoré sa na Taproot vzťahuje, je to, že z Taproot adres nie je možné spendovať pomocou ECDSA, nakoľko v Bitcoine používa Schnorrove podpisy. Toto obmedzenie však v custodial modeli nepredstavuje zásadný problém: systém môže držať prostriedky na legacy/SegWit adresách pod vlastnou kontrolou a zároveň bez obmedzení odosielať prostriedky aj na Taproot adresy zákazníkov, pričom spendovanie z týchto Taproot adres už zostáva na strane používateľa.

Následne sme vykonali prieskum existujúcich implementácií MPC/TSS v rôznych programovacích jazykoch a s odlišnými vlastnosťami. Medzi relevantné patrili napríklad:

1. open-source implementácie a knižnice spojené s Fireblocks (*mpc-lib / MPC-CMP*)
2. implementácia od TaurusHQ v jazyku Go (*taurusHQ/multi-party-sig*)
3. implementácia od Binance (*bnb-chain/tss-lib*)
4. a mnohé ďalšie z akademického prostredia či komunitné implementácie threshold ECDSA/Schnorr (napr. knižnice odvodené od CGGMP21/CGGMP24, implementácie z prostredia Dfns, a samostatné implementácie FROST pre Schnorr podpisy).

³ https://en.wikipedia.org/wiki/Threshold_cryptosystem

Zvolili sme implementáciu v jazyku **Rust**, najmä kvôli dôrazu na bezpečnosť (typová bezpečnosť, pamäťová bezpečnosť), lepšej integrácii do existujúcich interných systémov a vhodnosti pre budúce audity. Ďalším praktickým faktorom bolo aj nadviazanie komunikácie s maintainermi vybraného riešenia, čo naznačovalo aktívny vývoj knižnice a vyššiu pravdepodobnosť dlhodobej udržateľnosti.

Ako výsledok výskumu sme sa rozhodli použiť implementáciu protokolu z rodiny CGGMP, konkrétne variant označovaný ako **CGGMP24**, dostupný ako vetva v repozitári *LFDT-Lockness/cggmp21*. CGGMP24 predstavuje novšiu iteráciu veľmi blízku CGGMP21, no rieši identifikované bezpečnostné aspekty (<https://github.com/LFDT-Lockness/cggmp21/security>) a upravuje niektoré časti protokolu. Z pohľadu projektu nám poskytuje vhodný kompromis medzi praktickou použiteľnosťou, bezpečnostnými vlastnosťami a dostupnosťou v podobe open-source implementácie v Ruste.

Posledným krokom výskumu bolo overiť, či už existujú open-source projekty, ktoré by nad CGGMP24 budovali vyššiu vrstvu (napr. „wallet-like“ systém), čím by sa projekt mohol posunúť smerom k cieľovému typu riešenia bez potreby implementovať všetko od nuly. Výsledkom bolo, že sme identifikovali iba jedno existujúce demo, konkrétne projekt **m2ux/tss_demo**, ktorý ukazuje základné použitie knižnice v reálnom komunikačnom nastavení. Toto demo poskytlo užitočné informácie pri pochopení praktickej integrácie a tokov správ, avšak jeho parametre a funkcionality nezodpovedali požiadavkám nášho projektu (napr. zameranie na odlišné použitie a chýbajúce rozšírenia ako predgenerovanie presignatures či uzol s offline podpisovaním správ). Tento stav nás priviedol k rozhodnutiu navrhnuť a implementovať vlastné riešenie nad CGGMP24: jednoduché, zrozumiteľné a rozšíriteľné; v tomto prípade s demonštračnou aplikáciou, ktorá bude slúžiť ako otvorený základ pre MPC/TSS podpisovanie v custodial prostredí.

2.1. Presignatures

Počas výskumu knižnice sme identifikovali, že CGGMP24 podporuje mechanizmus **presignatures** (predpodpisy), ktorý umožňuje rozložiť proces threshold podpisovania na dve fázy: **predvýpočet** a **finálne podpísanie správy**. Presignature je kryptografický objekt, v ktorom sa vopred spoločne vygenerujú a „zafixujú“ všetky náhodné hodnoty a parametre potrebné pre budúci podpis, avšak bez znalosti konkrétnej správy, ktorá sa bude podpisovať. Výsledkom je, že každá strana získa vlastný lokálny presignature záznam (určený na jednorazové použitie) a zároveň existujú verejné pomocné údaje viazané na danú sadu presignatures, ktoré umožňujú neskôr korektne overiť a zložiť výsledný podpis.

Keď sa neskôr objaví správa na podpis, podpisovanie už nevyžaduje opätovné spúšťanie náročného protokolu: každá strana dokáže zo svojho presignature záznamu a zo správy

deterministicky vypočítať **partial signature**. Tieto partial signatures sa potom dajú jednoducho agregovať do štandardného ECDSA podpisu. Praktický efekt je, že „ťažká“ časť práce (komunikácia, náhoda, commitments...) sa presunie dopredu a samotný moment podpisu sa zmení na ľahkú lokálnu operáciu plus zber a kombináciu výsledkov.

Presignatures zároveň prinášajú dôležité bezpečnostné pravidlo: **sú striktne jednorazové**. Opakované použitie tej istej presignature sady na podpis rôznych správ vedie k úniku informácie o tajnom kľúči (analógia ku klasickému ECDSA problému s opakovaným nonce). Z toho dôvodu systém musí presignatures spravovať ako spotrebiteľný zdroj (pool) a garantovať, že každá sada bude použitá najviac raz.

Existencia presignatures v knižnici sa ukázala byť kľúčová pri vytváraní dizajnu a celkovej architektúry nášho prototypu.

3. Idea modelu

Keďže komerčné MPC riešenia pre podpisovanie blockchainových transakcií sú spravidla uzavreté (closed-source), verejne dostupné informácie nepostačujú na presný opis ich internej architektúry. Model navrhnutý v tomto projekte preto predstavuje len jedno z možných riešení, ako môže byť MPC podpisovanie integrované do custodial systému. Cieľom tejto kapitoly je načrtnúť tok dát a roly komponentov a zároveň ukázať, v čom sa MPC/TSS model líši od tradičného podpisovania jedným privátnym kľúčom. Pre jednoduchosť zanedbáme detaily ako formy verifikácie, validácie požiadaviek, autorizácie a ďalších bezpečnostných mechanizmov; tieto aspekty sa v reálnom produkčnom prostredí však samozrejme nachádzať musia.

3.1. Model s jedným privátnym kľúčom

V tradičnom modeli je podpisovanie centralizované. Interný používateľ alebo klient vytvorí požiadavku na transakciu. Systém transakciu zostaví do formátu príslušného blockchainu (napr. Bitcoin alebo Ethereum), pripraví údaje určené na podpis a odovzdá ich komponentu, ktorý disponuje privátnym kľúčom. Tento komponent transakciu podpíše a podpísaná transakcia sa odošle do siete blockchainu.

Nevýhodou tohto prístupu je centralizácia rizika: privátny kľúč musí existovať v celku v rámci jedného bezpečnostného bodu (procesu, stroja alebo zariadenia). Takýto bod sa prirodzene stáva single point of failure – kompromitácia prostredia, ktoré kľúč drží a používa, umožňuje útočníkovi vytvárať platné podpisy bez spolupráce ďalších strán.



Obr. 1: Single private key

3.2. Model s MPC/TSS sieťou

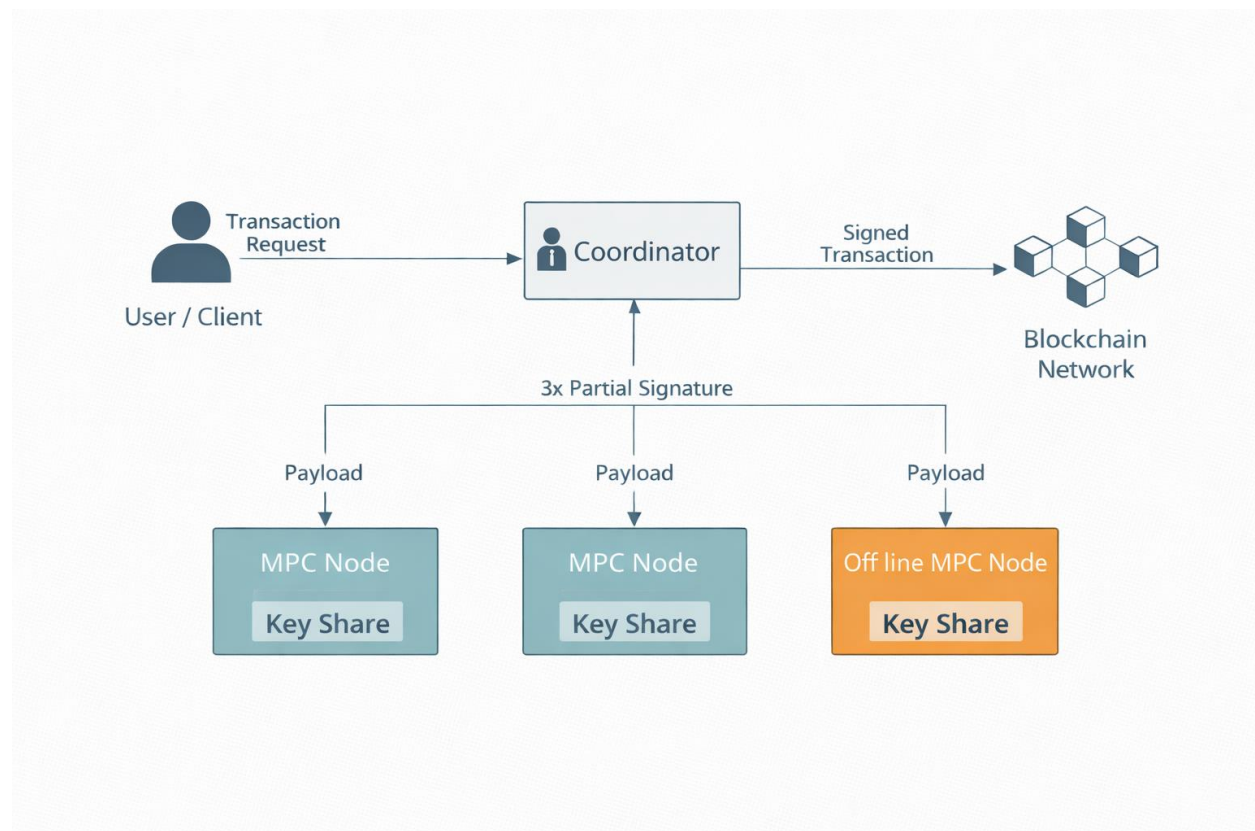
V navrhovanom modeli zostáva tok dát pred podpisovaním rovnaký: transakcia sa vytvorí a pripraví do formátu konkrétneho blockchainu. Rozdiel nastáva v okamihu podpisovania. Namiesto jediného uzla s kompletným privátnym kľúčom systém využíva sieť N MPC uzlov, kde každý uzol drží iba svoju časť tajomstva (key share) a je schopný prispieť k vytvoreniu výsledného podpisu. Komponent, ktorý prijíma požiadavky na podpis, nazývame **koordinátor**. Táto komponenta roz distribuuje požiadavku na podpis do MPC siete, vyžiada si od jednotlivých uzlov ich čiastkové podpisy a následne ich zloží do finálneho podpisu, ktorý je pre blockchain nerozoznateľný od štandardného „single-signature“ podpisu.

Hlavná výhoda spočíva v tom, že privátny kľúč sa nikdy nevyskytne v celku na jednom zariadení. Kompromitácia jednej časti systému preto sama o sebe nepostačuje na vytvorenie platného podpisu; útočník by musel prekonať ochranu viacerých nezávislých komponentov a získať dostatočný počet podielov na to aby dokázal ovplyvňovať podpisovací proces.

3.3. Offline uzol ako dodatočná bezpečnostná vrstva

Aby sme architektúru priblížili k praktickým custodial scenárom s vyššími bezpečnostnými nárokmi, model ďalej rozširujeme o **offline uzol** (air-gapped signer). Tento uzol je súčasťou MPC siete z pohľadu key share, avšak počas bežnej prevádzky nie je pripojený k internetu ani k internej sieti. Koordinátor teda priamo komunikuje len s online uzlami ($n-1$), no na zostavenie finálneho podpisu potrebuje aj časť z offline uzla.

V praxi to znamená, že podpisovaná transakcia (resp. dáta určené na podpis) sa do offline uzla prenesie fyzickým alebo izolovaným kanálom (napr. QR kód, USB médium a pod.). Offline uzol vykoná lokálny výpočet potrebný na vytvorenie svojho čiastkového podpisu a výsledok sa rovnakým offline kanálom prenesie späť do online prostredia. Koordinátor potom doplní chýbajúci čiastkový podpis k ostatným a zostaví finálny podpis transakcie, ktorá môže byť následne odoslaná do blockchainu. Takýto dizajn zvyšuje odolnosť voči kompromitácii online infraštruktúry tým, že časť podpisovacieho procesu vyžaduje fyzickú interakciu s izolovaným prostredím. Zopár presnejších detailov k bezpečnosti a dôvodu tohto návrhu si potom ešte rozoberieme v kapitole *Threat model*.



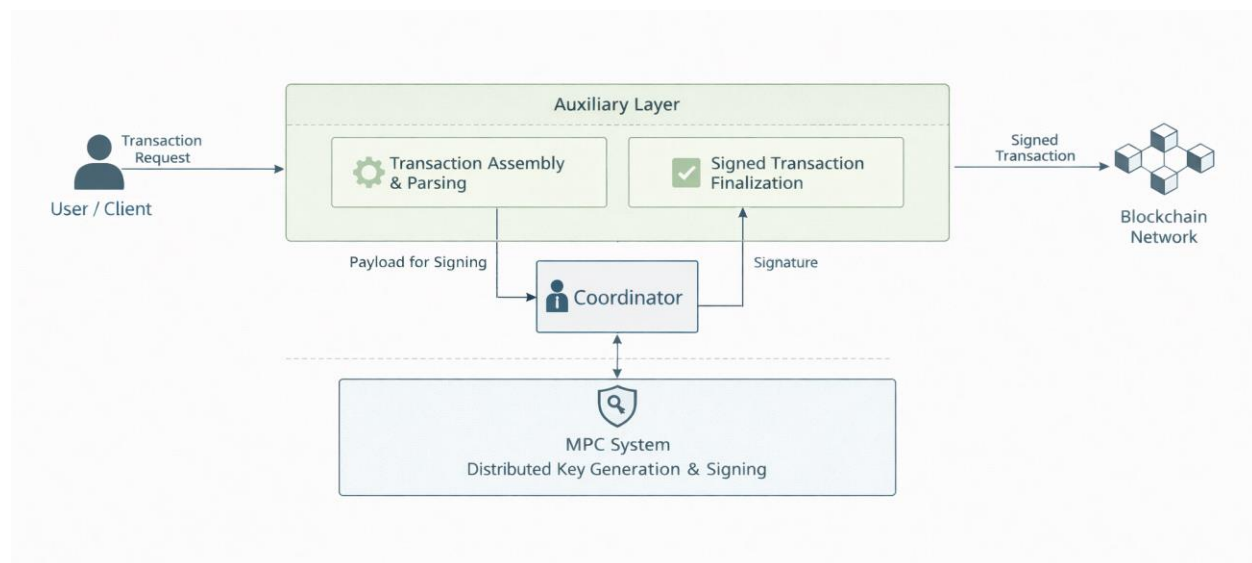
Obr. 2: MPC/TSS model

4. Architektúra

4.1. Higher-level architecture

Aby bolo možné tento model dosiahnuť v rozumnej a rozšíriteľnej podobe, architektúra je prirodzene rozdelená minimálne na dve vrstvy:

1. pomocná vrstva (auxiliary layer) zabezpečujúca transformáciu medzi formátmi konkrétnych blockchainových transakcií a dátami určenými na podpis
2. samotný MPC systém, ktorý realizuje distribuované generovanie kľúčov a vytváranie podpisov.



Obr. 3: Higher-level architecture with auxiliary layer

Je dôležité zdôrazniť, že MPC/TSS systém je z princípu iba „podpisovací blackbox“. Na rozhraní sa správa analogicky ako lokálne podpisovanie privátnym kľúčom: dostane vstupné dáta určené na podpis a vráti podpis. Samotný MPC protokol nerieši význam týchto dát – nevie, či ide o Bitcoin transakciu, Ethereum transakciu, alebo ľubovoľný iný reťazec bajtov. Zodpovednosť za to, aby sa podpisoval správny payload (napr. hash transakcie podľa pravidiel daného blockchainu) a aby bol výsledný podpis správne vložený späť do transakcie, patrí do pomocnej vrstvy. Práve toto rozdelenie umožňuje udržať MPC jadro jednoduché, formálne zrozumiteľné a použiteľné naprieč viacerými blockchainmi.

Pomocné vrstvy teda pokrývajú úlohy ako: zostavenie alebo parsovanie transakcie, výpočet presných dát určených na podpis (vrátane všetkých pravidiel konkrétneho chainu), odovzdanie týchto dát MPC systému a následné „finalizovanie“ transakcie prilepením výsledného podpisu do správneho formátu. Z hľadiska produkčného nasadenia ide o

nevyhnutnú časť systému, pretože len tak sa dá garantovať, že podpis zodpovedá reálnej a validnej transakcii daného blockchainu. Zároveň je to však do veľkej miery samostatný problém (doménová logika formátov transakcií), ktorý je oddeliteľný od core MPC logiky a môže byť implementovaný postupne a neskôr.

4.2. Auxiliary layer

Auxiliary layer tvorí pomocnú vrstvu systému, ktorá pracuje na úrovni blockchainových transakcií a prepája doménovú logiku konkrétneho chainu s MPC podpisovacím jadrom. Vo výslednom projekte bola táto vrstva implementovaná a slúži na demonštráciu funkčnosti celého prototypu na lokálnom Bitcoin blockchaine v režime **regtest**.

V aktuálnej implementácii auxiliary layer zostavuje špecifický typ Bitcoin transakcie (SegWit), pripraví z nej správne dáta určené na podpis a odovzdá ich MPC systému. Po získaní výsledného podpisu následne transakciu finalizuje a odošle ju do lokálneho blockchainového prostredia. Okrem toho táto vrstva zabezpečuje aj interakciu s regtest prostredím tak, aby bolo možné demonštrovať celý end-to-end tok systému: od inicializácie lokálneho blockchainu, cez vytvorenie transakcie a jej podpis MPC systémom, až po prijatie transakcie blockchainom. Podrobnejšie implementačné aspekty tejto vrstvy spoločne s regtest prostredím budú rozobraté v kapitole *Implementácia*.

Z architektonického hľadiska je auxiliary layer zároveň nositeľom multichainovosti. Samotný MPC systém podpisuje iba vstupné dáta a nerozlišuje, či ide o Bitcoin, Ethereum alebo iný formát. Zodpovednosťou auxiliary vrstvy je pripraviť pre konkrétny blockchain správny podpisový payload a po získaní podpisu z neho zostaviť validnú transakciu daného chainu. V našom prípade to znamená, že rovnaké MPC jadro je možné použiť pre blockchainya využívajúce ECDSA nad krivkou secp256k1, pričom rozdiely medzi jednotlivými chainmi sú izolované práve do tejto pomocnej vrstvy.

4.3. MPC system

MPC systém v tomto projekte predstavuje sieť podpisovacích uzlov, ktoré spoločne realizujú threshold podpisovanie bez toho, aby sa privátny kľúč v celku vyskytol na jednom zariadení. Na rozdiel od klasického „online MPC“ modelu, kde každý podpis vyžaduje interaktívnu komunikáciu medzi uzlami, prototyp je navrhnutý tak, aby v behovom prostredí minimalizoval interaktivitu a maximalizoval praktickú demonštrovateľnosť offline uzla.

Sú to práve presignatures, ktoré nám umožňujú navrhnúť to takýmto spôsobom a zároveň je to hlavný dôvod, prečo architektúru prototypu delíme na dve odlišné fázy: inicializačnú fázu v kontrolovanom prostredí a behovú fázu v oddelenom prostredí. V inicializačnej fáze

prebehnú všetky „ťažké“ výpočty, výsledkom ktorých sú prenositeľné množiny presignatures. V runtime fáze už uzly iba spotrebúvajú pripravené presignatures a lokálne vykonávajú výpočty partial signatures.

Pre účely demonštrácie a vývoja je zvolený minimalizovaný „**in-memory**“ **delivery model** s jednoduchým posielaním správ skrz lokálny kanál. Dôvodom je, že ľubovoľná konkrétna sieťová implementácia použitá v rámci prototypu (napr. WebSocket server) by aj tak nespĺňala nároky na produkčnú pripravenosť (autentizácia, autorizácia, šifrovanie, monitoring, retry/backpressure, odolnosť voči výpadkom). V dôsledku toho by bolo nutné takúto transportnú vrstvu pri reálnom nasadení prepracovať tak či tak. Minimalistický in-memory model preto umožňuje sústrediť čas a komplexitu na jadro projektu – návrh a implementáciu logiky uzlov, protokolov a doplnkových mechanizmov – bez toho, aby sa projekt zbytočne „zasekol“ na infraštruktúrnych detailoch.

Ako už bolo spomenuté v kapitole Intro, pri návrhu implementácie bol kladený dôraz na to, aby logika uzlov bola čo najľahšie prenositeľná na reálne nasadenie. To znamená, že uzly sú implementované tak, aby ich správanie a „životný cyklus“ bolo analogické reálnym komponentom: v produkcii by uzly mohli bežať ako samostatné procesy na fyzických strojoch, VM, v kontajneroch alebo iných izolovaných runtime prostrediach. V prototypovej verzii sú uzly reprezentované **asynchrónnymi Tokio taskmi**⁴, ktoré s koordinátorom komunikujú cez in-memory delivery. Ignorujúc výpadky siete a transportné chyby, tento model dobre odzrkadľuje charakter produkčného nasadenia: uzly prijímajú správy, aktualizujú svoj vnútorný stav a odosielajú ďalšie správy.

4.3.1. Inicializačná fáza

Inicializačná fáza prebieha v kontrolovanom (izolovanom) prostredí, v ktorom sú všetky podpisovacie uzly dostupné súčasne a môžu spolu komunikovať. Zámerom je realizovať všetky kryptograficky citlivé a komunikačne náročné operácie v prostredí, o ktorom máme vysokú mieru istoty, že je bezpečné. Tým sa znižuje riziko kompromitácie v okamihu, keď sa vytvárajú keyshares a presignatures, prakticky na nulu.

V tejto fáze prebiehajú najmä tieto kroky:

- generovanie pomocných informácií pre protokol (auxiliary information),
- distribuované generovanie kľúčov (key generation), ktoré vedie k vytvoreniu podielov kľúča pre jednotlivé uzly (key shares),

⁴ <https://docs.rs/tokio/latest/tokio/task/>

- predgenerovanie presignatures pre budúce podpisovanie.

Výstupom inicializačnej fázy sú serializované stavy, ktoré je možné preniesť do oddeleného behového prostredia. V prototypovej implementácii sa prenos realizuje prostredníctvom súborov (napr. cez USB médium), pričom z architektonického hľadiska ide o všeobecný export/import mechanizmus – rovnaký model umožňuje použiť aj iný prenosový kanál.

Dôležité je, že init fáza priamo produkuje množiny presignatures, ktoré sú pripravené na podpisovanie v runtime. Vďaka tomu v behovom prostredí nie je potrebné spúšťať komplexné protokoly na podpisovanie transakcií.

4.3.2. Behové prostredie (koordinátor + online uzly)

Behová fáza prebieha v samostatnom prostredí, ktoré reprezentuje produkčne bližší scenár. V tomto prostredí sa nachádza:

- **koordinátor**, ktorý prijíma požiadavky na podpis z pomocnej vrstvy a riadi tok podpisovania,
- **n-1 online uzlov**, ktoré sú pripojené na koordinátora a disponujú lokálnym presignature poolom importovaným z inicializačnej fázy.

V runtime už uzly nemusia vykonávať interaktívny MPC protokol. Po prijatí požiadavky na podpis každý online uzol:

- vyberie ďalšiu presignature zo svojho poolu,
- lokálne vypočíta partial signature pre dané dáta,
- odošle partial signature koordinátorovi.

Podmienka bezpečnosti je, že presignature sa nikdy nesmie použiť viac než raz; v prototypovej implementácii sa preto presignatures spravujú ako FIFO fronta a po použití sa z poolu odstráňa. Koordinátor zbiera partial signatures od online uzlov, no k finalizácii podpisu mu stále chýba podpis z offline uzla.

4.3.3. Offline uzol (air-gapped signer)

Súčasťou prototypu je aj samostatný program reprezentujúci offline uzol. Tento uzol je nasadený na oddelenom hardvéri a v runtime prostredí nekomunikuje po sieti s koordinátorom ani s online uzlami. Offline uzol má importovaný vlastný presignature pool z rovnakej inicializačnej fázy.

Keď je potrebné podpísať správu, dáta určené na podpis sú prenesené do offline uzla prostredníctvom offline kanála (v prototype formou súboru na USB médiu). Offline uzol z poolu spotrebuje jednu presignature, vypočíta partial signature a výsledok je rovnakým offline kanálom prenesený späť do behového prostredia. Koordinátor následne doplní chýbajúci partial signature a vykoná agregáciu do finálneho podpisu.

Použitý prenosový kanál (súbory/USB) predstavuje v prototype praktickú aproximáciu offline prenosu; rovnaký koncept je kompatibilný aj s inými alternatívami (napr. QR kódy).

4.3.4. Agregácia podpisu

Koordinátor po získaní všetkých potrebných partial signatures vykoná ich agregáciu do finálneho podpisu. Agregácia využíva verejné údaje prislúchajúce presignature sade (commitments), ktoré sú rovnaké pre všetkých účastníkov danej sady a umožňujú korektné zloženie podpisu aj jeho následnú verifikáciu. Výsledný podpis je odovzdaný pomocnej vrstve, ktorá ho vloží do transakcie a transakciu finalizuje.

4.3.5. Obmedzenia prototypu

Prototyp je navrhnutý ako demonštračný systém a neuvažuje najmä s týmito aspektmi:

- autentizácia a autorizácia požiadaviek smerom ku koordinátorovi a uzlom,
- šifrovanie a integrita transportu v online komunikácii,
- riešenie výpadkov uzlov, sieťových chýb, timeoutov a opakovaného doručovania,
- reštarty, rollbacky a desynchronizácia presignature poolov

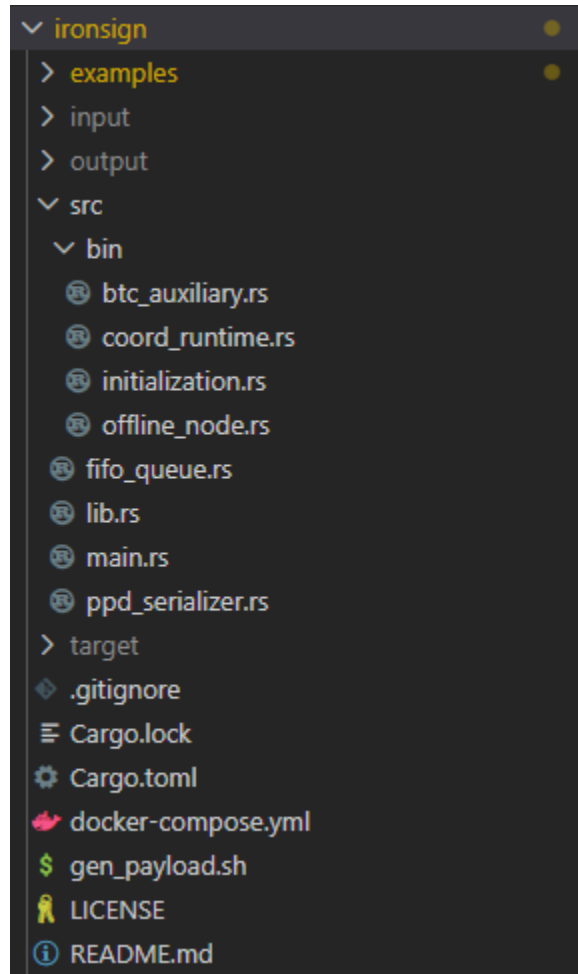
5. Implementácia

V tejto kapitole sa zameriavame na praktickú realizáciu navrhnutého systému. Cieľom nie je detailne komentovať jednotlivé bloky zdrojového kódu, ale skôr vysvetliť kľúčové implementačné rozhodnutia, štruktúru projektu a tie časti riešenia, ktoré sú z technického hľadiska najdôležitejšie alebo najzaujímavejšie. Predpokladáme, že samotný zdrojový kód je pre čitateľa dostupný a vzhľadom na jeho štruktúru a komentáre umožňuje samostatné doplnenie detailov, ktoré tu nebudeme explicitne rozoberať.

Kapitola je preto rozdelená na viacero tematických podkapitol. Najprv stručne opíšeme štruktúru projektu a rozdelenie jeho častí, následne sa budeme venovať jednotlivým komponentom implementácie.

5.1. Štruktúra

Projekt je z implementačného hľadiska rozdelený do troch hlavných častí. Prvú časť tvorí samotný **MPC systém**, ktorý je implementovaný v troch samostatných moduloch zodpovedajúcich trom vykonávacím režimom programu: **coord_runtime**, **initialization** a **offline_node**. Tieto moduly spolu pokrývajú celý životný cyklus systému – od prípravy stavov v kontrolovanom prostredí, cez online runtime prostredie s koordinátorom, až po samostatný offline podpisovací uzol.



Obr. 4: Compressed flowchart Initialization

Druhú časť tvorí modul **btc_auxiliary**, ktorý predstavuje pomocnú Bitcoin vrstvu. Jeho úlohou je práca s reálnymi Bitcoin transakciami v regtest prostredí: zostavenie transakcie, príprava podpisového payloadu pre MPC systém, finalizácia transakcie po získaní podpisu a interakcia s lokálnym blockchainovým prostredím. Tento modul tak prepája abstraktné MPC podpisovanie s konkrétnym použitím na úrovni Bitcoin transakcií.

Tretiu časť tvoria pomocné implementačné moduly. Modul **fifo_queue** slúži na správu presignatúr vo forme fronty. Hoci jazyk Rust poskytuje dátové štruktúry použiteľné na implementáciu FIFO správania (napr. `VecDeque`), v projekte bola vytvorená vlastná zjednodušená abstrakcia, ktorá obmedzuje operácie nad frontou len na tie, ktoré zodpovedajú nášmu bezpečnostnému modelu práce s presignatúrami. Cieľom je minimalizovať možnosť nesprávnej manipulácie s presignature poolom a zreteľnejšie vyjadriť, že presignatúry majú byť spotrebúvané striktne jednorazovo a v predpokladanom poradí.

Ďalším pomocným modulom je **ppd_serializer**, ktorý rieši serializáciu štruktúry `PresignaturePublicData`. Táto štruktúra je v použitej knižnici `CGGMP24` nevyhnutná pri agregácii partial signatures do finálneho podpisu, avšak samotná knižnica pre ňu neposkytuje priamo implementácie `Serialize` a `Deserialize`. Z tohto dôvodu bolo potrebné vytvoriť vlastný serializačný mechanizmus, aby bolo možné tieto dáta korektne ukladať, prenášať a znovu načítať v rámci navrhnutej architektúry.

Okrem samotných zdrojových modulov sa v repozitári nachádzajú aj ďalšie podporné súbory. Priečinok **examples** obsahuje experimentálne a testovacie programy, ktoré vznikali počas vývoja s cieľom lepšie pochopiť fungovanie knižnice `CGGMP24` a overiť čiastkové návrhové rozhodnutia. Súbor **gen_payload.sh** slúžil v skorších fázach vývoja na generovanie náhodného payloadu pre testovanie podpisovania; po implementovaní `btc_auxiliary` už však nie je potrebné pracovať iba s umelým payloadom, keďže systém dokáže generovať reálne podpisové dáta odvodené z Bitcoin transakcie. Súbor **docker-compose.yml** sa používa na spustenie lokálneho Bitcoin blockchainu v režime regtest vo forme Docker kontajnera, čím poskytuje izolované a reprodukovateľné testovacie prostredie pre end-to-end demonštráciu.

5.2. Regtest

Jedným z cieľov projektu bolo neukázať iba to, že MPC systém dokáže korektne podpisovať ľubovoľné vstupné dáta, ale aj to, že dokáže správne podpisovať dáta, pre ktoré je v praxi určený. V našom prípade ide o podpisové payloady odvodené z Bitcoin transakcií. Testovanie na reálnom Bitcoin blockchaine by však bolo pre potreby projektu zbytočne náročné, komplikovanejšie na správu a zároveň by vyžadovalo použitie reálnych finančných prostriedkov. Z tohto dôvodu sme sa rozhodli využiť lokálny Bitcoin blockchain v režime **regtest**.

Regtest predstavuje testovacie prostredie, v ktorom je možné spustiť Bitcoin blockchain lokálne a plne ho riadiť pomocou príkazov. To znamená, že je možné napríklad vytvárať nové

adresy, prideľovať im prostriedky, generovať ľubovoľný počet blokov alebo opakovane testovať rôzne scenáre bez závislosti od verejnej siete. Pre účely tohto projektu ide o ideálne riešenie, pretože umožňuje overovať správanie systému na reálnych Bitcoin transakciách, no zároveň odstraňuje praktické komplikácie spojené s testovaním na produkčnom blockchaine.

Použitie regtestu nám tak umožňuje demonštrovať end-to-end funkcionality celého riešenia: od vytvorenia transakcie, cez prípravu podpisového payloadu, jeho podpísanie MPC systémom, až po finalizáciu a prijatie transakcie blockchainom. V implementácii je regtest prostredie spúšťané lokálne vo forme Docker kontajnera, čo zároveň zjednodušuje reprodukovateľnosť experimentov a používanie projektu na rôznych strojoch.

5.3. BTC auxiliary

Ak sa pozrieme na obr. 3, pomocná vrstva by v ideálnom prípade mala zabezpečovať najmä prácu s transakciami: ich zostavenie, prípravu podpisového payloadu, finalizáciu po získaní podpisu a komunikáciu s MPC systémom prostredníctvom koordinátora. V produkčnom prostredí by sa táto vrstva prirodzene starala iba o doménovú logiku Bitcoin transakcií a o integráciu s podpisovacím jadrom. Samotný blockchain by fungoval nezávisle od systému a nebol by priamo „obsluhovaný“ aplikáciou.

V našom demonštračnom prototype má však `btc_auxiliary` o niečo širšie kompetencie. Okrem vyššie uvedených úloh zabezpečuje aj obsluhu lokálneho regtest blockchainu, napríklad vytváranie adresy, generovanie blokov a inicializáciu testovacích podmienok. Dôvodom je zjednodušenie prezentácie dema: namiesto toho, aby bolo potrebné blockchain obsluhovať manuálne alebo prostredníctvom ďalšieho samostatného programu, všetka logika súvisiaca s Bitcoin regtest prostredím je sústredená v jednej pomocnej vrstve. Keďže táto vrstva už aj tak pracuje s Bitcoin API a transakčnými formátmi, ide o prirodzený kompromis, ktorý zjednodušuje celkový control flow programu a zároveň nemení architektonickú podstatu MPC systému.

Z pohľadu funkčnej demonštrácie je dôležité ukázať, že systém nepodpisuje iba náhodné dáta, ale korektne pracuje s reálnou Bitcoin transakciou. Uvažujme adresy A, B a C, pričom adresa B je vytvorená z verejného kľúča poskytnutého MPC systémom. Na adresu A najprv v regtest prostredí pridáme prostriedky a následne z adresy A odošleme bitcoiny na adresu B. Tým dostaneme UTXO, ktoré je viazané na verejný kľúč MPC systému.

Následne zostrojíme novú transakciu, ktorá má odoslať prostriedky z adresy B na adresu C. Takáto transakcia môže byť blockchainom prijatá iba v prípade, že obsahuje platný podpis zodpovedajúci verejnému kľúču adresy B. Keďže adresa B bola odvodená z verejného kľúča

MPC systému, úspešné podpísanie takejto transakcie predstavuje dôkaz, že MPC systém efektívne disponuje podpisovacou schopnosťou zodpovedajúcou privátnemu kľúču tejto adresy.

Úloha `btc_auxiliary` v tomto procese spočíva v tom, že z transakcie pripraví správny podpisový payload, odovzdá ho MPC systému na podpis, po získaní výsledného podpisu ho vloží späť do transakcie a tú následne odošle do blockchainového prostredia. V poslednom kroku vygeneruje ďalšie bloky, aby sa transakcia stala potvrdenou a bolo možné jednoznačne overiť jej prijatie blockchainom. Ak blockchain transakciu akceptuje, demonštrácia tým potvrdzuje nielen správnosť samotného podpisu, ale aj to, že celý end-to-end tok – od vytvorenia transakcie, cez MPC podpisovanie, až po finalizáciu a odoslanie do siete – funguje korektne pre reálny Bitcoin scenár.

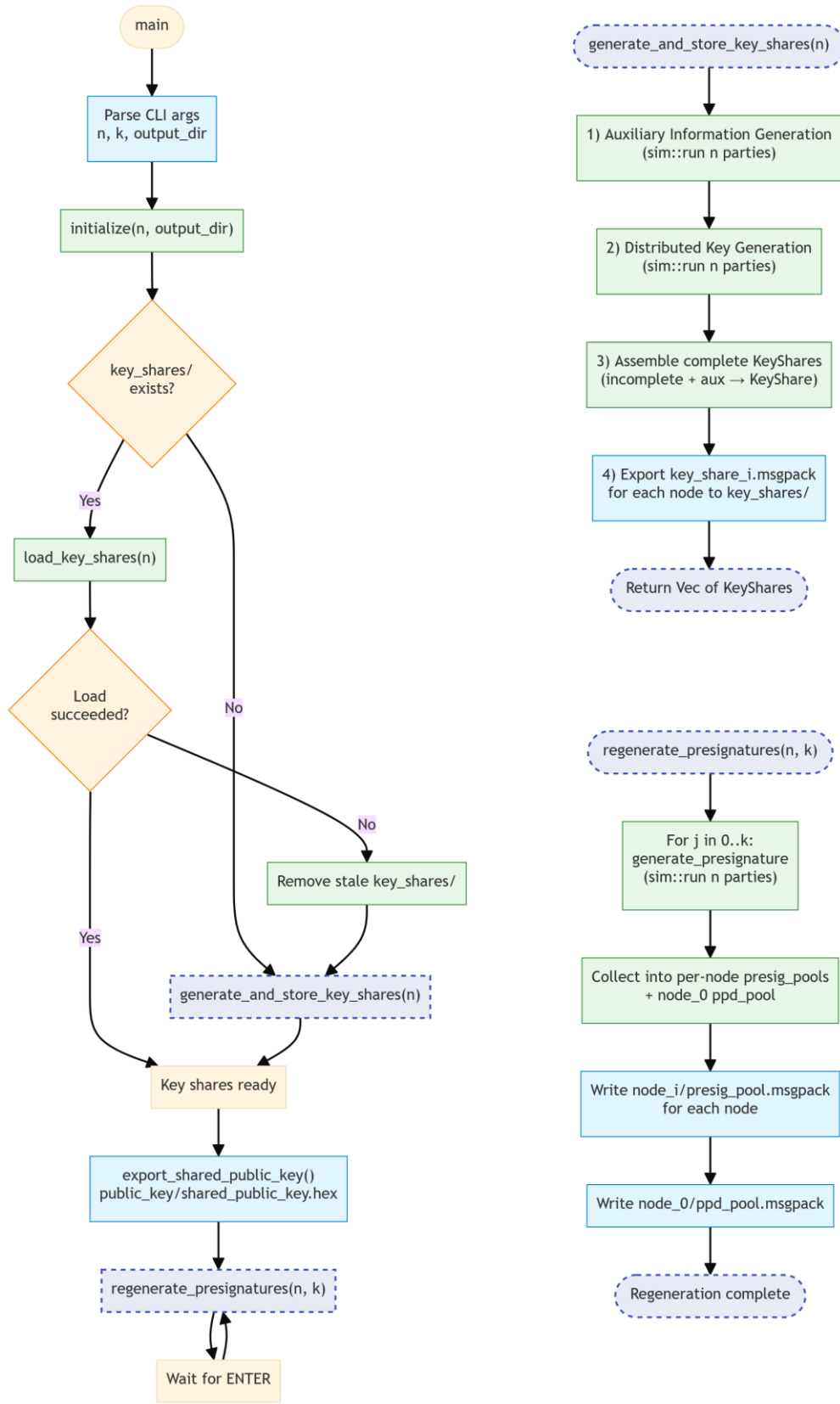
5.4. Initialization

Inicializačná fáza predstavuje samostatnú časť systému, ktorej úlohou je pripraviť všetky dáta potrebné pre následné podpisovanie v runtime prostredí. V klasickom modeli použitia CGGMP protokolu by jednotlivé uzly medzi sebou komunikovali po sieti, prechádzali by interaktívnymi fázami protokolu a postupne by si distribuovane vygenerovali pomocné dáta, key shares a neskôr aj presignatures. Počas návrhu projektu sme pôvodne vychádzali práve z tohto predpokladu a uvažovali sme o tom, že bude potrebné vytvoriť aspoň lokálnu „mini-sieť“ uzlov, ktoré budú spolu komunikovať a routingom správ simulovať reálnu distribuovanú prevádzku.

Postupne sa však ukázalo, že pre náš konkrétny use-case takýto model nie je potrebný. Keďže inicializačná fáza prebieha v kontrolovanom a dôveryhodnom prostredí, kde vieme garantovať bezpečnosť zariadenia, nie je nutné budovať ani sieťovú, ani lokálnu komunikačnú vrstvu medzi uzlami len kvôli tomu, aby sme získali výsledné stavy potrebné pre neskoršie podpisovanie. Kľúčovým zistením bolo, že knižnica `round-based`, na ktorej CGGMP24 stavia, poskytuje mechanizmus tzv. **simulovaného behu** (`round_based::sim`). Tento mechanizmus umožňuje na jednom zariadení lokálne nasimulovať viacero strán protokolu a ako výstup získať presne tie isté výsledné stavy, aké by vznikli pri skutočnom distribuovanom behu.

V praxi to znamená, že inicializačná fáza nemusí riešiť žiadny networking ani lokálny message routing. Namiesto toho sa na jednom zariadení spustí simulovaný beh pre všetky uzly naraz a výsledkom sú ich individuálne stavy. Tým sa výrazne znižuje implementačná komplexita aj počet riadkov kódu, pričom sa zároveň nemení kryptografický význam výsledku. Z pohľadu projektu išlo o jedno z najdôležitejších návrhových rozhodnutí, pretože

práve ono umožnilo oddeliť „ťažkú“ inicializačnú logiku od samotného runtime podpisovania a urobiť celý model podstatne jednoduchším a praktickejším.

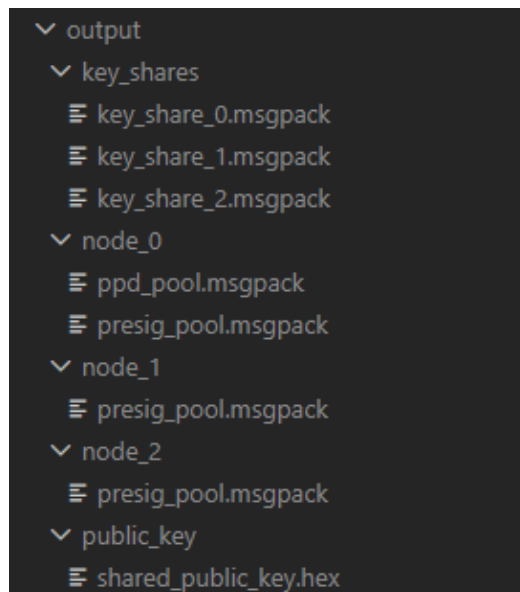


Obr. 5: Compressed flowchart - Initialization

Z kryptografického hľadiska prebieha inicializácia vo viacerých krokoch. Najprv sa pre každú stranu vygenerujú pomocné údaje (`aux_info`). Následne prebehne key generation, ktorého výstupom sú tzv. `incomplete_keyshare`. Až kombináciou `aux_info` a `incomplete_keyshare` vzniká finálny `key_share`. Práve z `key_share` je potom možné generovať presignatures, ktoré sa neskôr používajú pri podpisovaní. Dôležité je, že ak už raz máme finálny `key_share` uložený, nie je potrebné opakovane generovať `aux_info` ani `incomplete_keyshare`; pri ďalšom spustení inicializačnej fázy stačí existujúci `key_share` načítať a na jeho základe dopočítať nový pool presignatures.

Táto vlastnosť má praktický dôsledok aj na persistenciu stavu. Z pohľadu samotného runtime podpisovania postačuje, aby mal každý uzol k dispozícii iba svoj **presig_pool**. Naopak, z pohľadu obnovy systému alebo dogenerovania ďalších presignatures je potrebné uchovávať **key_share**. V našom návrhu preto initialization fáza vytvára a ukladá oba typy dát, avšak s odlišným účelom: presignatures sú určené na prenos do runtime prostredia a offline node, zatiaľ čo key shares slúžia ako podklad pre budúce doplnenie poolu a ako forma zálohy.

Implementačne sa pre každý uzol vytvára samostatný priečinok `node_i`, do ktorého sa serializuje jeho lokálny `presig_pool`. Uzol `node_0` je v našom modeli považovaný za offline uzol, a preto sa preň navyše pripravujú aj verejné údaje potrebné pri neskoršej agregácii podpisu, konkrétne `PresignaturePublicData` (v projekte označované skratkou `ppd`). Finálne `key_share` sa ukladajú oddelene do samostatného priečinka `key_shares`. Pri ďalšom spustení inicializačnej fázy sa systém najprv pozrie, či už priečinok `key_shares` existuje a obsahuje použiteľné kľúče. Ak áno, znovu ich použije a vygeneruje iba nový pool presignatures. Ak nie, spustí celý proces od začiatku.



Obr. 6: Príklad výstupu Initialization pre 3 uzly

Výsledkom inicializačnej fázy je teda sada serializovaných stavov pripravených na prenos do ďalších častí systému. Do runtime prostredia sa prenášajú pooly presignatures pre online uzly, do offline uzla pool presignatures aj ppd, a key shares zostávajú uložené ako lokálna záloha a základ pre prípadný ďalší regen presignatures. Ak si zoberieme za príklad situáciu z obr. 6, kde máme výstup pre 3 uzly, tak na PC2 budeme prenášať node_1 a node_2 kvôli runtime prostrediu, a zložku public_key kvôli btc_auxiliary. Do offline node zase presunieme node_0. Zložka key_shares nám ostane v Initialization outpute, pre potreby budúceho generovania.

Takýto model je zároveň praktický aj bezpečnostne výhodný: v runtime prostredí sa nemusia nachádzať dlhodobé key shares, ale iba spotrebovateľné presignatures určené na konkrétny počet budúcich podpisov.

5.5. Serialization

V projekte používame na serializáciu knižnicu **rmp-serde**, ktorá využíva formát **MessagePack**. Táto voľba bola motivovaná najmä dvoma dôvodmi. Po prvé, rmp-serde predstavuje stabilný a bežne používaný spôsob, ako v prostredí Rustu serializovať dátové štruktúry cez serde do kompaktného binárneho formátu. Po druhé, v čase implementácie už knižnica bincode, ktorá sa v Rust ekosystéme dlhší čas považovala za štandardnú voľbu pre binárnu serializáciu, nebola ďalej udržiavaná.

Použitie formátu MessagePack je pre náš prípad vhodné aj z praktického hľadiska. V porovnaní s textovým formátom, akým je napríklad JSON, ide o kompaktnejšie binárne reprezentovanie dát, čo je výhodné pri ukladaní a prenose stavových dát uzlov, najmä presignature poolov a ďalších štruktúr používaných v inicializačnej fáze. Keďže v našom modeli sa serializované dáta prenášajú medzi jednotlivými časťami systému prostredníctvom súborov, prioritou bola jednoduchá integrácia, spoľahlivá deserializácia a rozumná priestorová efektivita. `rmpe-serde` tieto požiadavky spĺňa.

5.6. Coordinator runtime

Runtime časť systému predstavuje simuláciu toho, ako by MPC podpisovacia infraštruktúra mohla fungovať v reálnom nasadení na viacerých zariadeniach. V našom prototype sú online uzly reprezentované ako asynchrónne tasky, pričom centrálnym riadiacim bodom je **koordinátor** (ktorý je sám asynchrónny task). Koordinátor prijíma vstupy z ostatných častí systému (najmä z `btc_auxiliary`), rozdistribuuje podpisové požiadavky online uzlom a po zozbieraní ich odpovedí vytvára výsledný podpis.

Pri štarte runtime prostredia koordinátor najprv vytvorí $n-1$ asynchrónnych taskov reprezentujúcich online uzly. Komunikácia medzi koordinátorom a týmito uzlami prebieha lokálne prostredníctvom `mpsc::unbounded_channel`. Uzly medzi sebou navzájom nekomunikujú; z pohľadu runtime prostredia je postačujúce, aby každý uzol prijímal príkazy od koordinátora a vracal mu svoj výsledok. Táto architektúra zodpovedá tomu, že v runtime fáze už neprebieha interaktívny MPC protokol v pôvodnom zmysle, ale iba lokálne spotrebúvanie vopred pripravených presignatures.

Každý online uzol podporuje dva základné príkazy, reprezentované ako enum: **Init** a **Sign**. Príkaz **Init** slúži na načítanie lokálneho stavu uzla z príslušného priečinka a jeho inicializáciu. V praxi to znamená načítanie serializovaného `presig_pool` do pamäte. Po úspešnej inicializácii uzol vráti koordinátorovi veľkosť svojho poolu, čím koordinátor overí, že všetky online uzly boli načítané korektne a že majú k dispozícii rovnaký počet presignatures. Zjednodušenie prototypu spočíva v tom, že ak sa pri inicializácii objaví chyba, celý proces sa opakuje pre všetky uzly naraz; granulárna obnova jedného konkrétneho uzla v deme podporovaná nie je.

Po úspešnej inicializácii sú online uzly pripravené na podpisovanie. Na to slúži príkaz **Sign**, ktorým koordinátor pošle každému uzlu podpisový payload. Uzol následne spotrebuje jednu presignature zo svojho lokálneho poolu a vráti koordinátorovi svoju **PartialSignature**. Koordinátor teda po tomto kroku disponuje $n-1$ online partial signatures. Po dodaní

chýbajúcej partial signature z offline uzla následne vykoná finálnu agregáciu a uloží výsledný podpis do výstupného priečinka.

Runtime časť zároveň podporuje aj praktický scenár vyčerpania presignature poolu. Pri inicializácii koordinátor zistí veľkosť poolov všetkých online uzlov a túto hodnotu si uchováva. Následne si vedie počet už vykonaných podpisov a vie tak detegovať moment, keď sa pool vyčerpá. Vtedy je potrebné vygenerovať nové presignatures v initialization programe, preniesť ich do runtime prostredia a znovu spustiť `Init`, ktorý načíta nový stav uzlov. Tým sa systém vráti do stavu pripraveného na ďalšie podpisovanie.

Dôležitou implementačnou vlastnosťou runtime časti je aj parameter **--prehashed**. Ten určuje, či sa vstupný payload má považovať už za hotové zahashované dáta, alebo či sa má hashovanie vykonať až vo vnútri MPC systému. Bez tohto parametra sa používa štandardná cesta `DataToSign::digest::<Sha256>(payload)`, teda payload sa najprv zahashuje. Pri použití auxiliary vrstvy však do MPC systému neposúvame ľubovoľné vstupné dáta, ale už konkrétny podpisový payload odvodený z Bitcoin transakcie, ktorý je z pohľadu ECDSA už hashom pripraveným na podpis. V takom prípade by ďalšie hashovanie v MPC systéme viedlo k nesprávnemu podpisu. Parameter **--prehashed** preto umožňuje spracovať payload priamo ako prehashed vstup bez dodatočného hashovania. Naopak, pri testovaní na náhodných dátach alebo pri všeobecnom payload režime sa tento parameter nepoužíva a hash sa vytvára až vo vnútri MPC systému.

5.7. Offline node

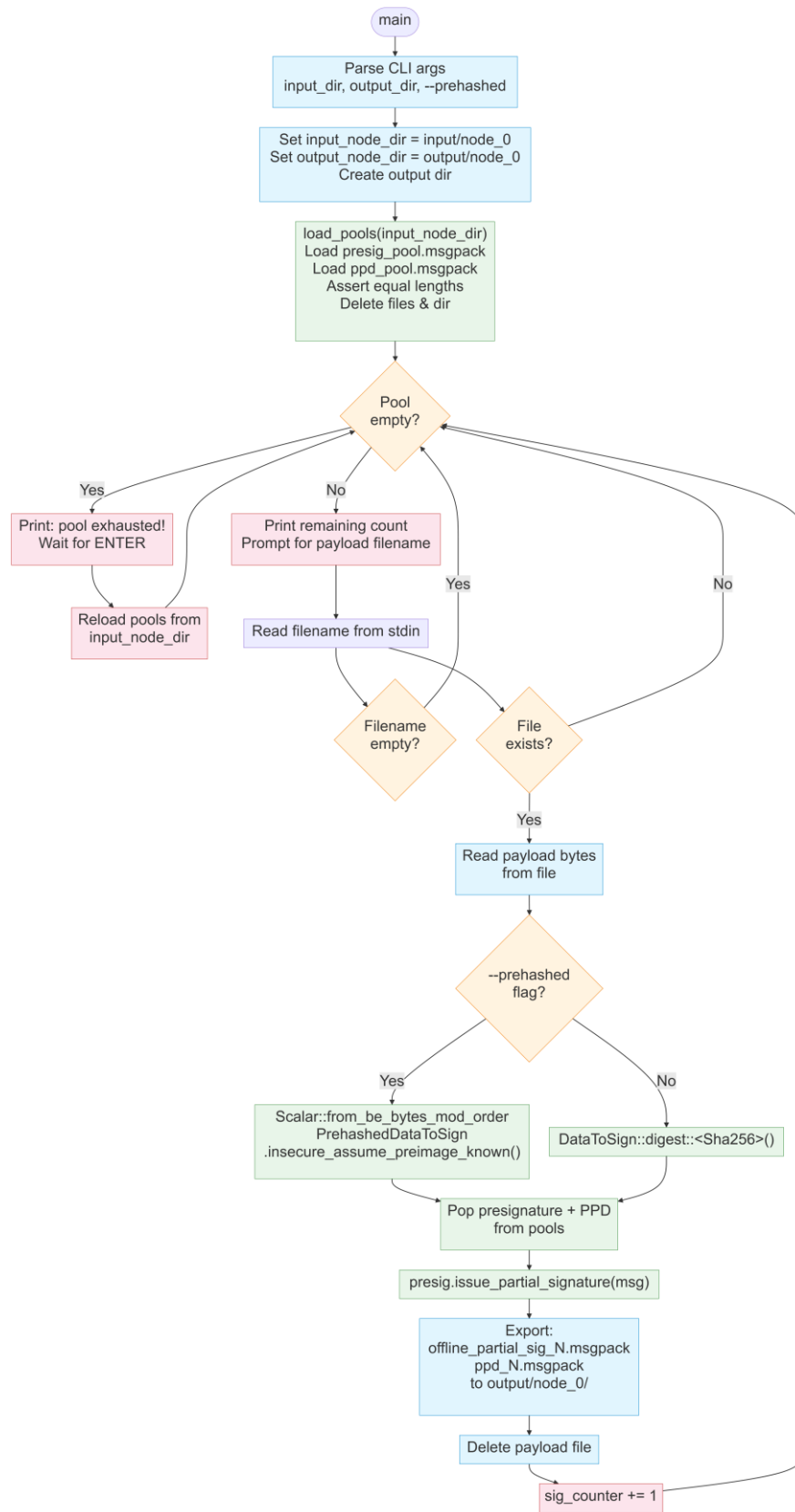
Offline node je samostatný program určený na prevádzku na oddelenom zariadení, ktoré nie je pripojené k internetu ani k online časti systému. Jeho úlohou je reprezentovať poslednú, manuálne obsluhovanú podpisovaciu stranu v n-of-n modeli a tým vytvárať dodatočnú bezpečnostnú bariéru voči kompromitácii online infraštruktúry.

Z pohľadu správania funguje offline node podobne ako online uzly v runtime prostredí: aj on pracuje s vlastným lokálnym `presig_pool`, z ktorého pri podpise spotrebuje jednu presignature a vytvára z nej **PartialSignature** pre zadaný payload. Na rozdiel od online uzlov však nekomunikuje cez interný komunikačný kanál s koordinátorom, ale je ovládaný manuálne prostredníctvom vstupných a výstupných súborov.

Pri inicializácii offline node načíta zo svojho priečinka lokálny `presig_pool`, ktorý bol pripravený v initialization fáze. Okrem neho sa pre offline uzol pripravuje a ukladá aj samostatná FIFO štruktúra obsahujúca **PresignaturePublicData** (ppd). Tieto dáta sú potrebné pri neskoršej agregácii podpisu, pretože finálny podpis sa vytvára pomocou funkcie `combine(partial_signatures, commitments, m)`, kde parameter

commitments zodpovedá práve PresignaturePublicData patriacemu k použitej sade presignatures.

Dôležitou vlastnosťou offline node je jeho **manuálna obsluha**. Payload sa doň dostáva offline kanálom (v našom prototype prostredníctvom súborov prenášaných cez USB médium), používateľ spustí podpisový krok lokálne na offline zariadení a výsledok následne tým istým offline kanálom prenesie späť do runtime prostredia. Týmto spôsobom sa zabezpečuje, že posledný chýbajúci podiel na podpise nevzniká v online infraštruktúre a jeho vytvorenie vyžaduje fyzickú interakciu s izolovaným zariadením.



Obr. 7: Flowchart - Offline node

6. Očakávaný priebeh demonštrácie

Cieľom demonštrácie je ukázať end-to-end podpisovanie v modeli MPC/TSS s jedným offline uzlom. Demonštrácia je navrhnutá ako systém rozdelený na tri hlavné časti – **inicializáciu**, **runtime prostredie** a **offline podpisovanie** – doplnené o **auxiliary vrstvu**, ktorá zabezpečuje prácu s Bitcoin transakciami. Medzi jednotlivými časťami systému sa prenášajú iba serializované dáta, a to buď lokálne v rámci jedného počítača, alebo prostredníctvom offline kanála. V našom prototype je týmto offline kanálom prenos súborov cez USB médium.

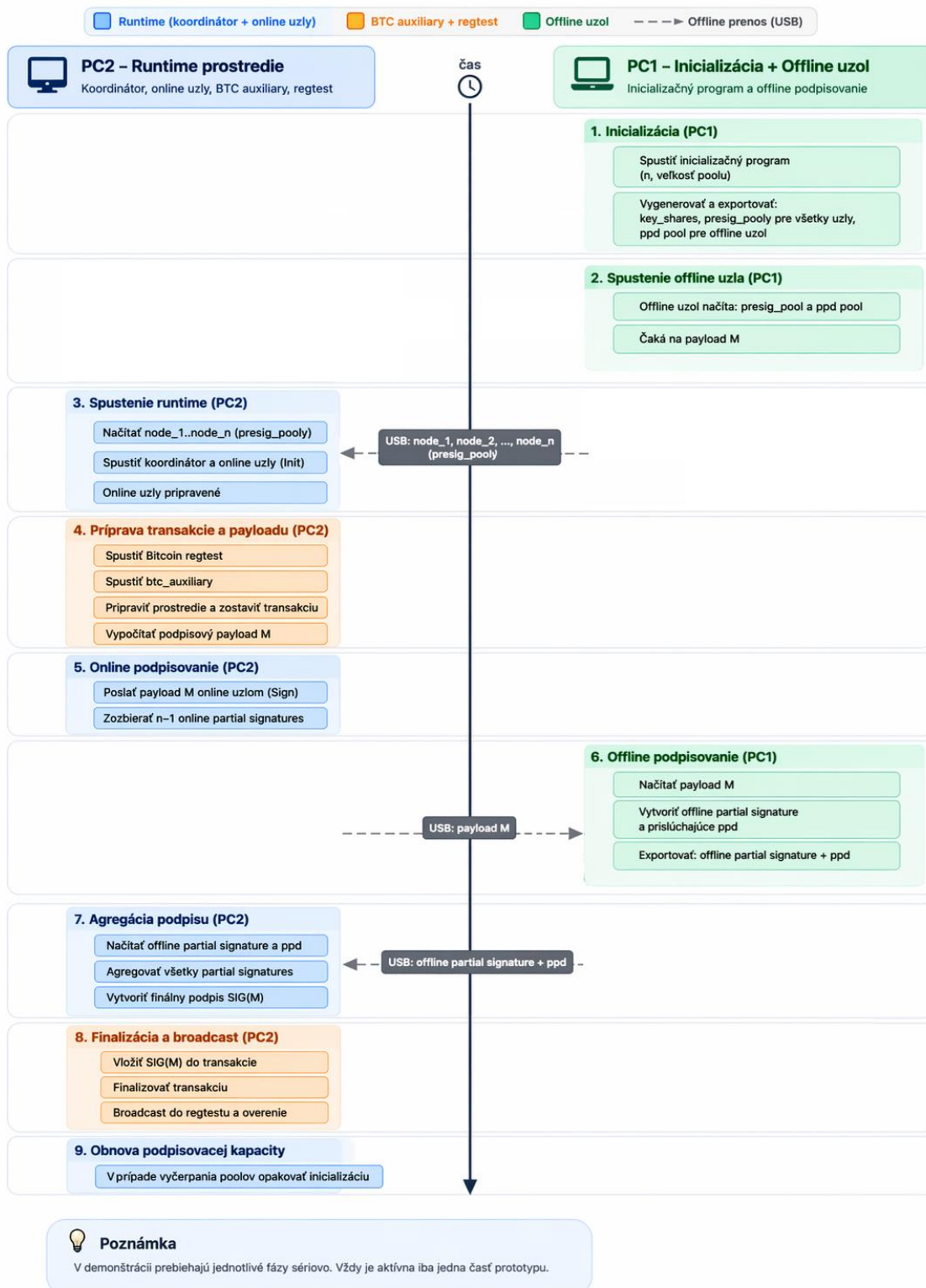
Táto kapitola opisuje očakávaný priebeh demonštrácie už na úrovni konkrétnej implementácie, teda nie iba ako všeobecný architektonický návrh, ale ako reálny tok operácií medzi jednotlivými časťami programu. Cieľom je ukázať, ako sa systém inicializuje, ako sa pripraví dáta potrebné na podpisovanie, ako prebieha komunikácia medzi koordinátorom a online uzlami a akým spôsobom sa do procesu zapája offline uzol.

6.1. Základné predpoklady

Použitý podpisový režim je **n-of-n** (non-threshold), teda na vytvorenie finálneho podpisu sú potrebné partial signatures od všetkých zúčastnených uzlov. Počet online uzlov je v prototype variabilný, keďže sú reprezentované ako asynchrónne Tokio tasky. Offline uzol je vždy **presne jeden** a predstavuje samostatnú, manuálne obsluhovanú podpisovaciu stranu.

Priebeh demonštrácie

MPC/TSS podpisovanie s jedným offline uzlom



Obr. 8: Priebeh demonštrácie

6.2. Priebeh

0. Nastavenie demonštrácie

0.1 Pripraviť dva počítače:

- **PC1:** inicializačné prostredie + program offline uzla
- **PC2:** runtime prostredie s koordinátorom + auxiliary vrstva + Bitcoin regtest

0.2 Pripraviť offline prenosový kanál (v deme prostredníctvom súborov na USB).

1. Inicializačná fáza (PC1)

1.1 Spustiť inicializačný program v izolovanom prostredí s parametrami:

- počet uzlov
- veľkosť poolu presignatures

1.2 Overiť, či už existujú uložené key_share:

- **ak áno:** použiť ich znovu
- **ak nie:** vygenerovať ich od začiatku

1.3 Ak sa key_share generujú od začiatku, prebehnú kroky:

- vygenerovanie aux_info
- vygenerovanie incomplete_keyshare
- spojenie aux_info + incomplete_keyshare -> key_share

1.4 Následne sa pre všetky uzly vygenerujú:

- **FIFO pooly presignatures**
- pre offline uzol aj **FIFO množina ppd** (PresignaturePublicData)

1.5 Serializovať výsledné dáta do súborov:

- pre každý uzol vytvoriť priečinok node_i
- do každého node_ uložiť príslušný presig_pool
- node_0 považovať za **offline uzol**

- do node_0 uložiť navyše aj FIFO štruktúru ppd
- všetky key_share uložiť osobitne do priečinka key_shares

1.6 Inicializačnú časť ponechať dostupnú pre prípadné budúce dogenerovanie presignatures.

2. Spustenie offline uzla (PC1)

2.1 Spustiť program offline uzla.

2.2 Offline uzol načíta zo svojho priečinka:

- presig_pool
- FIFO štruktúru ppd

2.3 Offline uzol prejde do režimu “čakám na payload na podpis”

2.4 Offline uzol nekomunikuje po sieti, iba cez:

- vstupné a výstupné súbory
 - offline prenosový kanál (USB)
-

3. Spustenie runtime prostredia (PC2)

3.1 Preniesť cez USB z PC1 na PC2 exporty online uzlov:

- priečinky node_1, node_2, ..., node_n

3.2 Spustiť runtime prostredie na PC2.

3.3 Koordinátor:

- Vytvorí n-1 online uzlov ako Tokio tasky
- pošle im príkaz **Init**

3.4 Každý online uzol po prijatí Init:

- načíta svoj presig_pool
- inicializuje sa

- vráti koordinátorovi veľkosť svojho poolu

3.5 Koordinátor overí:

- že všetky online uzly boli inicializované úspešne
 - že všetky pooly majú rovnakú veľkosť
-

4. Spustenie Bitcoin časti (PC2)

4.1 Spustiť lokálny Bitcoin blockchain v režime **regtest**.

4.2 Spustiť `btc_auxiliary`.

4.3 Auxiliary vrstva:

- pripojí sa k regtestu
- pripraví lokálne blockchainové prostredie
- vytvorí adresy a testovacie podmienky
- zostaví konkrétnu **SegWit transakciu**
- z nej vypočíta podpisový payload M

4.4 Auxiliary vrstva odovzdá payload M koordinátorovi.

5. Online podpisovanie (PC2)

5.1 Koordinátor rozdistribuuje payload M do online uzlov za pomoci príkazu **Sign**.

5.2 Každý online uzol:

- vezme jednu presignature z lokálnej množiny
- vytvorí partial signature pre payload M
- odošle partial signature koordinátorovi

5.3 Koordinátor zozbiera $n-1$ online partial signatures a prejde do režimu „chýba offline podpis“.

6. Offline podpisovanie (PC1)

6.1 Preniesť payload M z PC2 na PC1 cez offline kanál (USB/súbor)

6.2 Offline uzol:

- načíta payload M ktorý treba podpísať
- spotrebuje jednu presignature z lokálnej množiny
- vezme zodpovedajúce ppd z FIFO štruktúry
- vytvorí chýbajúci partial signature z offline node
- uloží výsledok do súboru
- do osobitného súboru uloží takisto commitment (ppd) pre túto sadu presignatures

6.3 Preniesť výsledný partial signature spolu s ppd späť na PC2.

7. Agregácia podpisu (PC2)

7.1 Koordinátor načíta:

- offline PartialSignature
- príslušné ppd

7.2 Koordinátor vykoná agregáciu:

- doplní offline partial signature
- spojí n partial signatures
- použije ppd
- vytvorí finálny podpis $SIG(M)$

7.3 Koordinátor uloží výsledný podpis do výstupného priečinka.

8. Finalizácia transakcie a odoslanie do blockchainu (PC2)

8.1 Auxiliary vrstva:

- vloží podpis $SIG(M)$ späť do BTC transakcie
- finalizuje transakciu
- odošle ju do lokálneho Bitcoin regtestu

8.2 Následne sa overí, že blockchain transakciu prijal

9. Obnova podpisovacej kapacity

9.1 Koordinátor si eviduje:

- veľkosť poolu pri inicializácii
- počet už vykonaných podpisov

9.2 Ak sa pool vyčerpá:

- v initialization na PC1 sa vygenerujú nové presignatures
- exportujú sa do priečinkov uzlov
- prenesú sa do runtime prostredia
- znovu sa vykoná Init

9.3 Analogicky pre offline node

9.4 Tým sa systém vráti do stavu pripraveného na ďalšie podpisovanie

Poznámka (presignatures): Inicializačná časť bude po úvodnej inicializácii a vygenerovaní presignatures bežať ďalej (indefinitely) ako „zdroj“ pre prípadné neskoršie dopĺňanie presignatures. Ak sa v runtime prostredí presignatures spotrebujú, koordinátor vyžiada ich dogenerovanie v inicializačnej časti a nové presignatures sa následne opäť vyexportujú (v deme cez súbory/USB) do runtime a offline uzla. Ide o vedomé obmedzenie prototypu a jeho dizajnu: dopĺňanie presignatures vyžaduje spätný krok do inicializačného prostredia. V praxi je však možné generovať veľké množstvo presignatures dopredu, čím sa potreba dopĺňania výrazne znižuje; zároveň ide o otvorenú otázku pre budúce zlepšenie, ako tento mechanizmus navrhnúť robustnejšie a efektívnejšie.

6.3. Overenie správnosti

Demonštrácia bude považovaná za úspešnú, ak:

- koordinátor získa partial signatures od všetkých online uzlov,
- offline uzol vytvorí a vráti svoju partial signature spolu s príslušným ppd,
- koordinátor úspešne agreguje finálny podpis,
- auxiliary vrstva finalizuje transakciu,
- a výsledná transakcia je akceptovaná Bitcoin regtestom.

7. User guide

Dostupná v GitHub repozitári prototypu. Vzhľadom k tomu, že repozitár je privátny, je potrebné povoliť prístup k repozitáru, čo samozrejme pre účely školského projektu bude umožnené.

8. Threat model

V tomto prototype uvažujeme útočníka, ktorý sa pokúša získať kontrolu nad digitálnymi aktívami buď kompromitáciou softvéru a infraštruktúry (online prostredie), alebo fyzickým prístupom k zariadeniam. Základná motivácia použitia MPC/TSS je odstránenie „single point of failure“ (SPOF) pri správe privátneho kľúča: žiadny uzol nedrží celý privátny kľúč v jednom mieste, a preto kompromitácia jedného stroja sama o sebe nemá viesť k exfiltrácii kľúča.

8.1. Kompromitácia koordinátora (SPOF na úrovni autorizácie)

Aj keď MPC rozkladá riziko úniku privátneho kľúča, koordinátor zostáva kritickým komponentom z hľadiska *autorizácie* podpisovania. Koordinátor je bod, ktorý prijíma požiadavky na podpis (napr. transakcie) a distribuuje payload na podpisovanie do MPC siete. Ak útočník kompromituje koordinátora, vie iniciovať podpisovanie ľubovoľných dát a pokúsiť sa tak vytvoriť škodlivú transakciu bez toho, aby priamo kompromitoval podpisujúce uzly.

Toto riziko v prototype mitigujeme práve zavedením jedného *offline uzla*, ktorého účasť je v režime n-of-n nevyhnutná na vytvorenie finálneho podpisu. Offline uzol vyžaduje manuálnu obsluhu: payload určený na podpis je prenesený offline kanálom (napr. súbor na USB), na offline zariadení je manuálne skontrolovaný a až následne je vygenerovaná partial signature. Ak payload nezodpovedá autorizovanej požiadavke (napr. transakcia nie je očakávaná, whitelisting/blacklisting adries alebo nesesí s požiadavkou používateľa), offline uzol podpis odmietne. Tým sa kompromitácia koordinátora sama o sebe nestáva postačujúcou na ukradnutie prostriedkov.

8.2. Prečo nestačí jedno offline podpisové zariadenie bez MPC

Mohlo by sa zdať, že ak offline uzol slúži ako autorizačná „brzda“, postačovalo by mať jediné offline zariadenie s celým privátnym kľúčom. Takéto riešenie však vytvára nový single point of failure: pri fyzickej kompromitácii offline zariadenia by útočník získal plnú kontrolu nad všetkými prostriedkami. Kombinácia online MPC a offline uzla rozkladá riziko: útočník musí súčasne realizovať úspešný útok na online infraštruktúru (aby vedel iniciovať a koordinovať

podpisovanie) aj fyzický útok (aby získal kontrolu nad offline časťou procesu). Táto kombinácia významne zvyšuje náročnosť útoku oproti modelu s jediným zariadením.

8.3. Umiestnenie ppd a jeho bezpečnostné dôsledky

V CGGMP24 pri finálnej agregácii podpisu nestačia iba partial signatures; funkcia `combine` navyše vyžaduje aj zodpovedajúce `PresignaturePublicData` (ppd). Tieto údaje sú viazané na konkrétnu sadu presignatures a bez nich nie je možné výsledný podpis korektne zostaviť.

Pre jednu konkrétnu sadu presignatures je ppd rovnaké pre všetky uzly. Z toho dôvodu nedáva zmysel ukladať ho duplicitne pri každom online uzle spolu s jeho lokálnym `presig_pool`, keďže by to iba zbytočne zväčšovalo objem prenášaných a ukladaných dát.

V našom prototype preto ppd uchováваме pri **offline uzle**. Toto rozhodnutie vychádza z toho, že initialization fáza aj offline node bežia na tom istom dôveryhodnom zariadení. ppd tak nemusí byť exportované do runtime prostredia spolu s online stavmi; postačuje, ak ho offline uzol pri podpise vráti koordinátorovi spolu s príslušnou partial signature. Koordinátor následne použije toto ppd pri finálnej agregácii podpisu.

Takéto riešenie je pamäťovo úspornejšie, zjednodušuje správu stavových dát a zároveň nijak nezvyšuje attack surface modelu nášho prototypu. V inom nasadení by bolo možné ppd uchovávať aj inde, pokiaľ by bolo dostupné v okamihu agregácie a zároveň by boli zachované požadované bezpečnostné vlastnosti prostredia.

9. Záver

Práca na tomto projekte priniesla zaujímavé výsledky a predstavovala hodnotnú skúsenosť z akademického aj praktického hľadiska. Počas riešenia bolo potrebné skombinovať teoretický výskum v oblasti MPC/TSS protokolov s návrhom reálnej architektúry a následnou implementáciou funkčného prototypu. Projekt tak nebol iba implementačným cvičením, ale aj procesom postupného spresňovania modelu, overovania návrhových rozhodnutí a hľadania rozumného kompromisu medzi bezpečnosťou, jednoduchosťou a praktickou použiteľnosťou.

Z požiadaviek na TSS podpisovanie sme naplnili všetky podstatné body: kľúč je rozdelený na N častí uložených na nezávislých uzloch, pričom žiadny uzol nedisponuje kompletným privátnym kľúčom, a podpis vzniká kooperáciou viacerých strán. Zároveň je prototyp navrhnutý multichainovo v rámci prieniku ECDSA nad `secp256k1`, a teda je použiteľný pre

Bitcoin (legacy/SegWit) aj Ethereum, s perspektívou budúceho rozšírenia podľa možnosti zvolenej podpisovej schémy.

Rovnako sme architektonicky navrhli očakávania kladené na podpisovací protokol samotný. Pri komunikácii medzi uzlami sa nikdy neodovzdávajú privátne kľúče, ale iba čiastkové hodnoty potrebné na vytvorenie podpisu. Návrh zároveň podporuje online aj offline podpisovanie – offline uzol je v modeli obsluhovaný manuálne a podpisuje požiadavky mimo online prostredia. Výsledkom je štandardný single-sig podpis, ktorý sa na blockchaine javí rovnako ako podpis vytvorený jedným privátnym kľúčom, t. j. bez rozpoznateľnej „multisig stopy“.

Okrem toho sa podarilo implementovať aj pôvodne len doplnkovú auxiliary vrstvu, vďaka ktorej je možné demonštrovať funkčnosť prototypu na reálnej, hoci lokálnej, Bitcoin infraštruktúre v režime regtest. Projekt tak neostáva iba pri podpise náhodných dát, ale ukazuje korektné fungovanie celého end-to-end toku od zostavenia transakcie, cez prípravu podpisového payloadu a MPC podpisovanie, až po finalizáciu a prijatie transakcie blockchainom.

Na pomery softvérových projektov sa pravdepodobne jedná o projekt s menším rozsahom z pohľadu celkového počtu riadkov kódu. Nepovažujeme to však za nedostatok, ale skôr za prirodzený dôsledok typu riešeného problému. V oblasti kryptografie, bezpečnosti a auditovateľnosti je žiaduce smerovať k čo najjednoduchšiemu a najprehľadnejšiemu riešeniu, ktoré minimalizuje zbytočnú komplexitu. Podobne ako významná časť práce bola venovaná výskumu problematiky, časť úsilia smerovala aj do refaktORIZÁCIE a zjednodušovania implementácie tak, aby bol výsledný kód čo najčitateľnejší, najkompaktnejší a čo najlepšie auditovateľný. Aj z tohto pohľadu preto považujeme zvolený výsledok za správny a primeraný cieľom projektu.

Na záver môžeme konštatovať, že výsledná implementácia zodpovedá cieľom stanoveným v dokumentácii projektu a predstavuje konzistentný, funkčný a ďalej rozšíriteľný základ pre MPC/TSS podpisovanie v custodial prostredí.

9.1. Ďalšie možné smerovanie

Hoci výsledný prototyp spĺňa hlavné ciele projektu a demonštruje funkčné MPC/TSS podpisovanie vrátane offline uzla a Bitcoin auxiliary vrstvy, prirodzene otvára aj viacero možností ďalšieho rozvoja.

Najdôležitejšou oblasťou ďalšieho vývoja je autorizácia podpisovania. Samotné MPC rieši distribúciu kľúča a znižuje riziko jeho kompromitácie, no samo osebe nerozhoduje, ktoré

požiadavky na podpis sú legitímne. Pri produkčnom nasadení je preto nutné doplniť mechanizmus, ktorý by nad koordinátorom vynucoval pravidlá podpisovania alebo nejakú formu viacúrovňového schvaľovania.

Prirodzeným smerom rozšírenia je aj robustnejšia správa presignatures a stavových dát. V prototype sme zámerne zvolili jednoduchý model založený na FIFO pooloch, pretože je pre demonštráciu prehľadný a dobre zodpovedá očakávanému control flow. V budúcnosti by však bolo vhodné doplniť bezpečnejšie spravovanie presignatures, kontrolu desynchronizácie štruktúr pracujúcich s presignatures, obnovu po páde alebo zvoliť úplne iný prístup využívajúci adresovateľné spravovanie presignatures s pomocou explicitných identifikátorov (napr. Hash mapa alebo nejaká forma databázy).

Významný priestor na ďalší vývoj ponúka aj prechod z lokálne simulovaného runtime modelu na reálne oddelené uzly. V súčasnom prototype sú online uzly reprezentované ako asynchrónne tasky, bez šifrovanej komunikácie a bez poriadných recovery mechanizmov. Produkčná verzia však toto všetko musí mať vyriešené, a teda, uzly reprezentované oddelenými procesmi alebo zariadeniami, autentizovanú komunikáciu medzi komponentmi, monitoring a odolnosť voči výpadkom.

Samostatnú oblasť ďalšieho rozvoja predstavuje aj offline uzol a jeho workflow. V prototype je offline interakcia realizovaná prostredníctvom súborov a USB média, čo je vhodné pre demonštračné účely. Do budúcnosti by však bolo možné uvažovať o robustnejšom formáte request/response dát, prípadne o alternatívnych offline kanáloch, napríklad už spomínaných QR kódach.

Prirodzeným pokračovaním by bol aj rozvoj auxiliary vrstvy. V súčasnej implementácii slúži na demonštráciu Bitcoin SegWit scenára v regtest prostredí, čím poskytuje praktický dôkaz funkčnosti celého prototypu. Do budúcnosti by ju však bolo možné rozšíriť o auxiliary vrstvu pre Ethereum alebo iné kryptomeny, čím by sa ešte jasnejšie ukázal multichain charakter navrhnutého MPC jadra.

Napokon, z bezpečnostného a prevádzkového hľadiska by bolo vhodné doplniť aj ďalšie mechanizmy, ktoré sú v prototype zámerne zjednodušené, napríklad šifrovanie uložených dát, kontrolu integrity exportov a importov, bezpečné mazanie spotrebovaných presignatures z disku alebo rozšírené testovanie systému.

Na základe toho možno povedať, že výsledný prototyp nepredstavuje uzavreté riešenie, ale funkčný a overený základ, na ktorom je možné ďalej stavať. Práve v tomto zmysle projekt napĺňa nielen demonštračný, ale aj výskumno-praktický cieľ: ukazuje, že navrhnutý model funguje, a zároveň vytvára priestor pre jeho ďalší technický a bezpečnostný rozvoj.